

# **09. Facetas y programación**

## **Ciencia de datos para la toma de decisiones I**

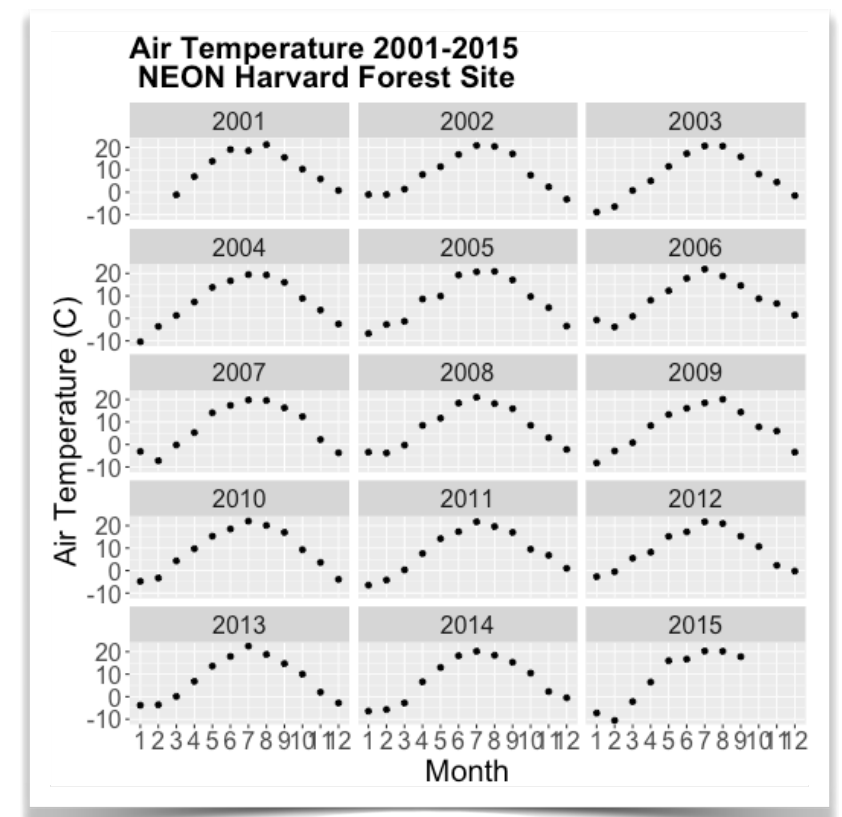
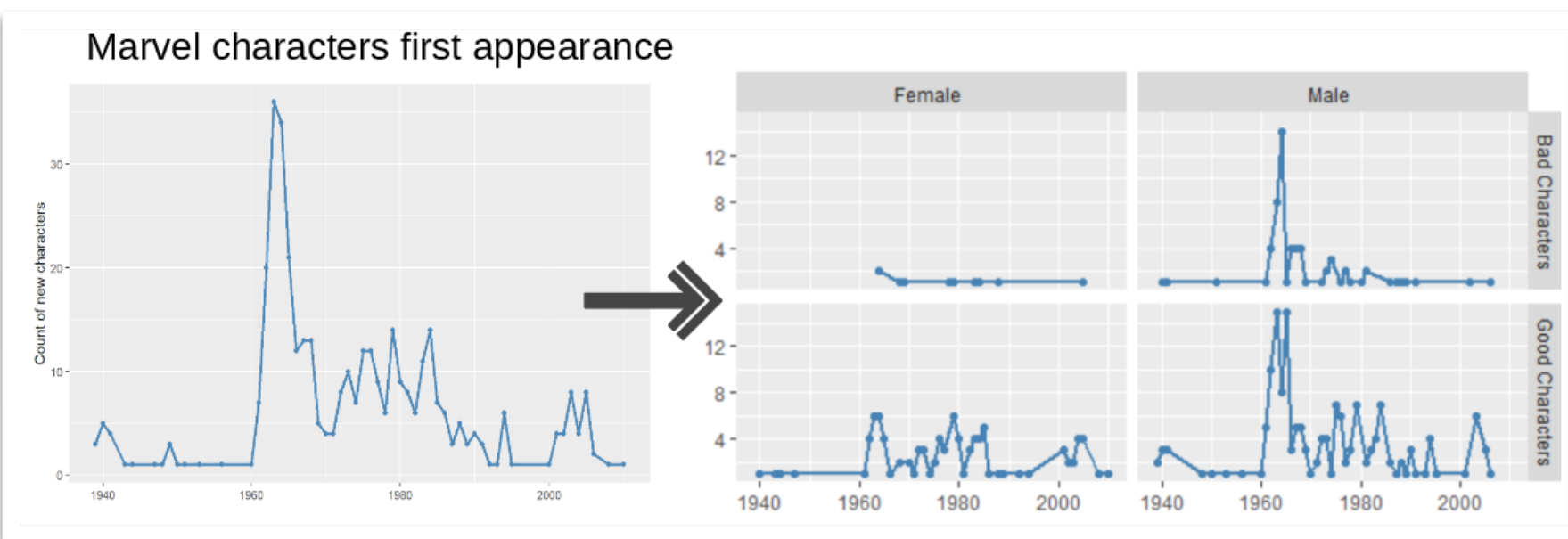
**Jorge  
Juvenal  
Campos Ferreira**

✉ [juvenal.campos@tec.mx](mailto:juvenal.campos@tec.mx)

# Facetas en R

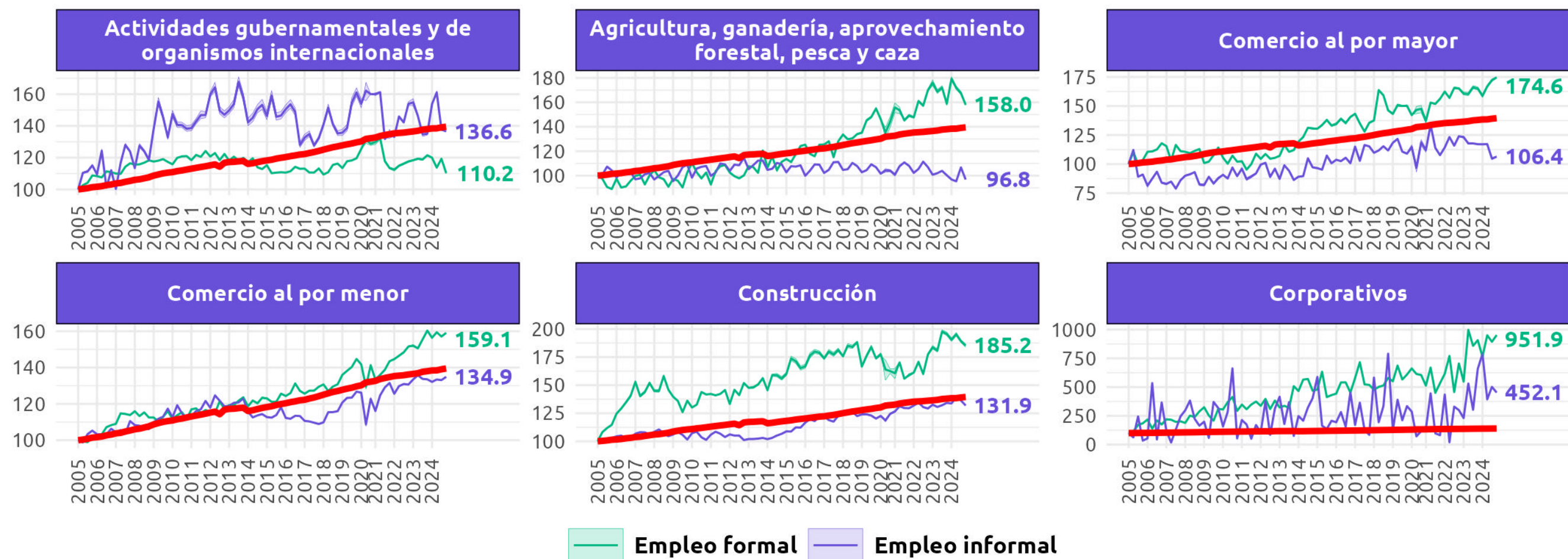
Las facetas en ggplot2 son una forma de **dividir un gráfico en múltiples paneles**, donde cada panel muestra un subconjunto de los datos según los valores de una o más variables categóricas.

La idea viene del concepto de "small multiples" de Edward Tufte: **en lugar de meter todo en un solo gráfico saturado**, generas una cuadrícula de gráficos pequeños que comparten los mismos ejes y escalas, facilitando la comparación.



# Evolución de los niveles de empleo **formal** e **informal** contra crecimiento de la población, por sector del SCIAN

Nivel 100 = 2005T1. Línea **roja** indica el crecimiento poblacional de los mayores de 15 años



**ELABORADO POR MÉXICO, ¿CÓMO VAMOS? CON DATOS DE LA ENOE**

# facet\_wrap

La función que nos permite pasar de un gráfico normal de ggplot a un gráfico de facetas. **facet\_wrap()** toma una sola variable y genera paneles que se van acomodando en filas como si fueran texto (de izquierda a derecha, saltando de línea).

Es ideal cuando **tienes una variable con varios niveles y no necesitas una estructura de cuadrícula específica**. Por ejemplo, `facet_wrap(~entidad)` te genera un panel por cada estado. Puedes controlar cuántas columnas o filas quieres con los argumentos `ncol()` o `nrow()`.

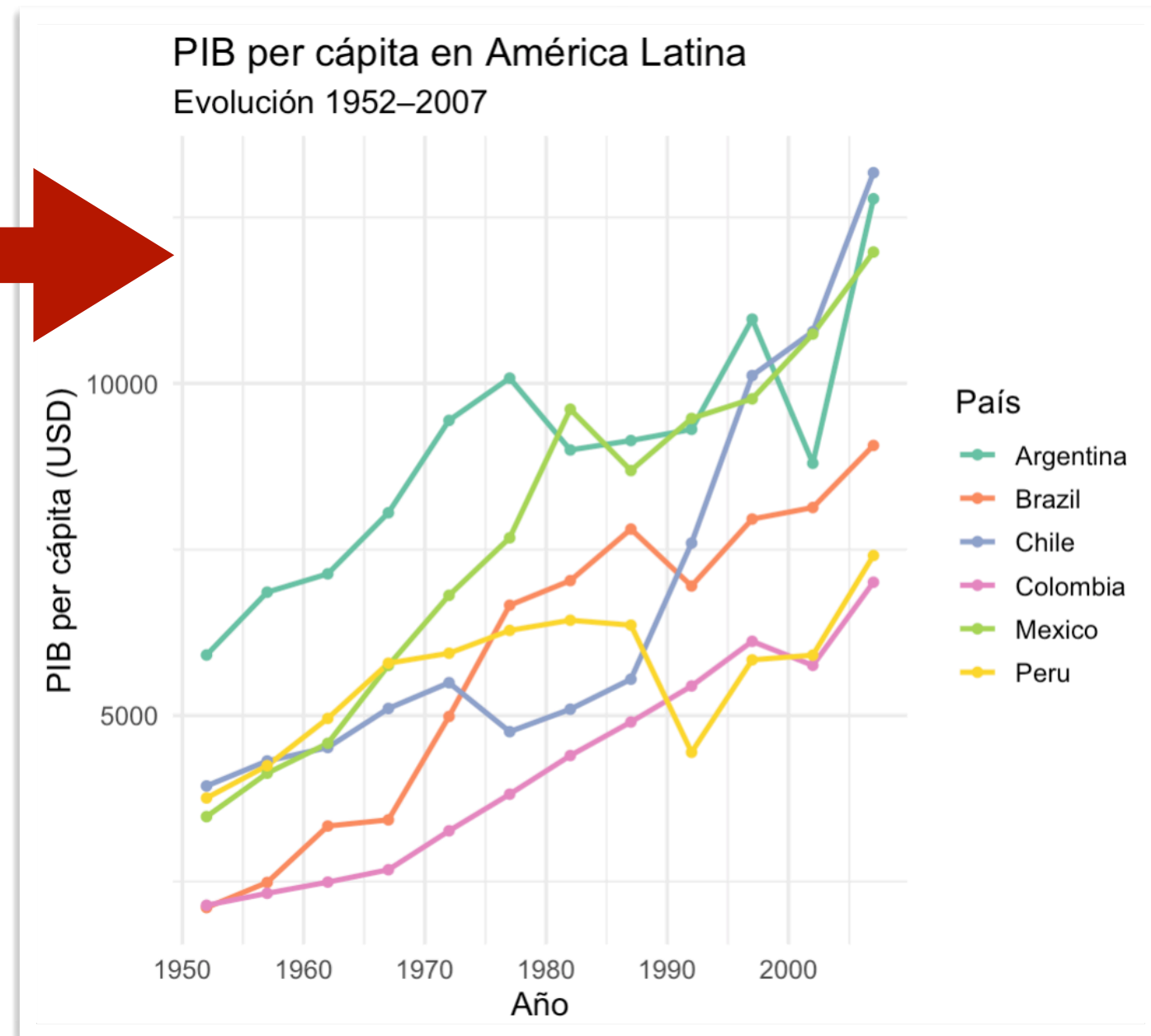
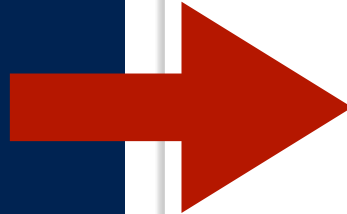
# Sin facet\_wrap

```
library(tidyverse)
library(gapminder)

## Filtrar algunos países de América Latina
países_latam <- c("Mexico", "Brazil", "Argentina",
                  "Colombia", "Chile", "Peru")

datos <- gapminder %>%
  filter(country %in% países_latam)

## Gráfica de espagueti
ggplot(datos,
  aes(x = year, y = gdpPercap,
      color = country)) +
  geom_line(linewidth = 1) +
  geom_point(size = 1.5) +
  scale_color_brewer(palette = "Set2") +
  labs(
    title = "PIB per cápita en América Latina",
    subtitle = "Evolución 1952-2007",
    x = "Año",
    y = "PIB per cápita (USD)",
    color = "País"
  ) +
  theme_minimal(base_size = 13)
```





# Con facet\_wrap

```
## Misma gráfica, ahora con un panel por país
ggplot(datos,
       aes(x = year, y = gdpPercap,
           color = country)) +
  geom_line(linewidth = 1) +
  geom_point(size = 1.5) +
  scale_color_brewer(palette = "Set2") +
  facet_wrap(~country, ncol = 3, scales = "free_y") +
  labs(
    title = "PIB per cápita en América Latina",
    subtitle = "Evolución 1952–2007 | Un panel por país",
    x = "Año",
    y = "PIB per cápita (USD)"
  ) +
  theme_minimal(base_size = 13) +
  theme(
    axis.text.x = element_text(angle = 90, hjust = 0.5),
    legend.position = "none",
    strip.text = element_text(face = "bold")
  )
```

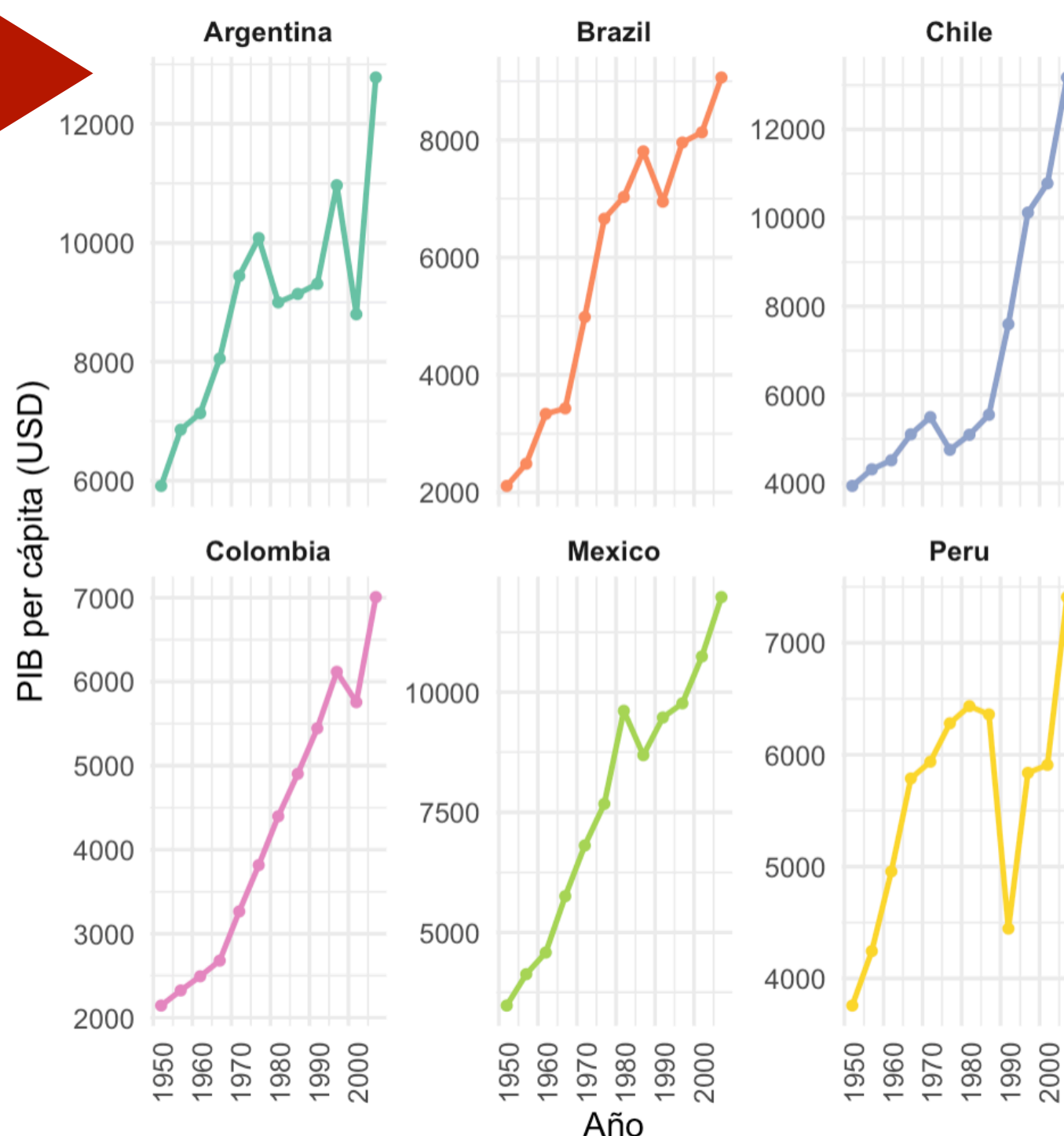
## Argumentos del facet\_wrap:

**~country:** La columna con las categorías que van a dar pie a cada faceta individual.

**ncol:** El número de columnas que va a tener la gráfica.

**scales = "free\_y":** libera los valores del eje "y".  
"free\_x" libera el "x" y "free" libera ambos ejes.

PIB per cápita en América Latina  
Evolución 1952–2007 | Un panel por país



## Ejercicio práctico 01.

Dados los datos de la carpeta de ejercicios prácticos, por favor, descargue los datos. La carpeta contiene las proyecciones de población de CONAPO de las entidades federativas a 2070.

1. Filtre los datos para que no aparezcan las proyecciones nacionales (`cve_ent != "00"`).
2. Grafique, en una sola gráfica de ggplot, las líneas de las proyecciones estatales. Discuta por qué a este tipo de gráficos se les conoce como gráficas de espagueti
3. Utilice la función *facet\_wrap()* para obtener una gráfica de facetas donde cada rectángulo muestre la proyección poblacional para cada uno de los 32 estados de México.
4. Guarde la gráfica en su carpeta de visualizaciones.
5. Si tuviera que generar una gráfica individual para cada estado, en vez de las facetas, **¿qué tendría que hacer?**

# Motivación: El problema del copiar-pegar

## Sin programación

- 32 estados = 32 scripts idénticos o script con 32 secciones repetidas
- Cambio pequeño = editar 32 archivos/secciones
- No escala a otros tipos de datos (como los datos municipales)
- Imposible mantener o modificar

## Con programación

- \* Una función, 32 parámetros
- \* Cambia lógica en un lugar
- \* Escala a cualquier cantidad
- \* Reproducible y profesional



# Operadores de comparación

Comparan valores y devuelven TRUE/FALSE

**==**

Igual a

**!=**

No igual a

**>**

Mayor que

**<**

Menor que

**>=**

Mayor o  
igual

**<=**

Menor o  
igual

## Ejemplos:

5 == 5    # TRUE

5 != 3    # TRUE

edad < 30    # depende del valor

ingresos >= 100000    # depende del valor

# Operadores lógicos

Combinan condiciones para tomar decisiones más complejas

&

AND: ambas verdaderas

|

OR: al menos una verdadera

!

NOT: invierte resultado

%in%

IN: valores **dentro** de un conjunto

## Ejemplos:

`(edad > 18) & (edad < 65) # Adulto en edad laboral`

`(región == "Norte") | (región == "Sur") # Dos regiones: la "Norte" y la "Sur"`

`!(ingresos < 50000) # "Quédate" con los ingresos que sean >= 50000`

`entidad %in% c("Puebla", "Jalisco", "Morelos") # Quédate con los estados que sean "Puebla", "Jalisco" y "Morelos"`

`!(entidad %in% c("Puebla", "Jalisco", "Morelos")) # Quédate con los estados que NO sean "Puebla", "Jalisco" y "Morelos" (se invirtió el resultado).`

# Condicionales if/else

Hay veces que queremos que un proceso se lleve a cabo cuando se cumple una condición lógica, y hay veces que queremos que otro código se ejecute cuando esta no se cumple.

## Para más de dos opciones

```
if (condición1) {  
  código1  
} else if (condición2) {  
  código2  
} else {  
  código3  
}
```

## Clasificar niveles de pobreza:

```
if (ingresos < 50000) {  
  nivel <- "extrema"  
} else if (ingresos < 100000) {  
  nivel <- "moderada"  
} else {  
  nivel <- "baja"  
}
```

## Nota:

Los condicionales evalúan SOLO la primera condición verdadera.

# Bucles *for*

## Repite código un número fijo de veces

```
for (i in 1:5) {  
  print(i)  
}
```

## Sumar elementos de un vector:

```
valores <- c(10, 20, 30)  
suma <- 0  
for (val in valores) {  
  suma <- suma + val  
}  
print(suma) # 60
```

## Bucle con índice:

```
for (i in 1:nrow(datos)) {  
  if (datos$pobreza[i] > 0.5) {  
    print(str_c("Estado", i, "en pobreza"))  
  }  
}
```

# Funciones propias

## Empaquetar código lógico para reutilizar

```
nombre_funcion <- function(argumento1, argumento2, ... argumento_n) {  
  # Cuerpo de la función  
  
  # ... proceso a guardar ...  
  
  resultado <- argumento1 + argumento2  
  return(resultado)  
}  
  
nombre_funcion(5, 3) # Llamar función
```

## Ejemplo: función para clasificar región

```
clasificar_región <- function(región) {  
  if (región %in% c("Chiapas", "Oaxaca")) {  
    return("Sur")  
  } else if (región %in% c("Sonora", "Chihuahua")) {  
    return("Norte")  
  } else {  
    return("Centro")  
  }  
}
```



# Estructura completa de las funciones

## Elementos de una función bien diseñada:

### Nombre descriptivo

calcular\_media\_pobreza() no x()

### Argumentos y parámetros

Parámetros que recibe con valores por defecto

### Validación

Verificar que los argumentos sean válidos

### Documentación

Comentarios explicando qué hace

### return()

Explicitar el resultado que la función debe devolver

## Ejercicio práctico 02.

Dados los datos de las proyecciones de población del ejercicio práctico número 01, ahora realice lo siguiente:

1. Genere el código para obtener solamente la gráfica de la proyección de población de **“Aguascalientes”**
2. Identifique las partes de ese código que cambiarían si ahora quisiera hacer la gráfica de **“Morelos”**
3. Genere una función que le permita construir la gráfica para cualquier estado, cambiando el estado seleccionado.
4. Utilice un bucle ***for*** para hacer las gráficas de los 32 estados del país.
5. ¿Qué pasa si le pone a la función: estado seleccionado == “Guadalajara”? ¿Funciona la función? ¿Qué podríamos hacer al respecto?